



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Ejercicio Práctico - Snake

Curso: PROGRAMACIÓN II – Sección: B

Docente: Ing. Breno Luque Zuñiga

ALUMNO: Yordan Antonio Cohaila Mamani

INTRODUCCIÓN

El presente trabajo consiste en el diseño e implementación de un videojuego tipo Snake utilizando el lenguaje de programación C# y una interfaz gráfica desarrollada en Windows Forms. El proyecto tiene como finalidad aplicar los principios de la Programación Orientada a Objetos (POO), integrando conceptos como clases, objetos, encapsulamiento, constructores, métodos, interfaces y colecciones dentro de una aplicación interactiva.

El videojuego desarrollado permite controlar una serpiente que se desplaza dentro de un tablero, recolectando alimentos para aumentar su tamaño y acumular puntos, mientras evita colisionar contra los bordes o contra su propio cuerpo. Además, el sistema incluye funcionalidades como detección de colisiones, control mediante teclado, reinicio de partida, control de puntaje y mensajes de victoria o derrota.

Mediante este proyecto se busca fortalecer las habilidades de programación lógica, manejo de eventos, desarrollo de interfaces gráficas y organización modular del código, permitiendo comprender de manera práctica la aplicación de la Programación Orientada a Objetos en el desarrollo de softwa

OBJETIVO GENERAL

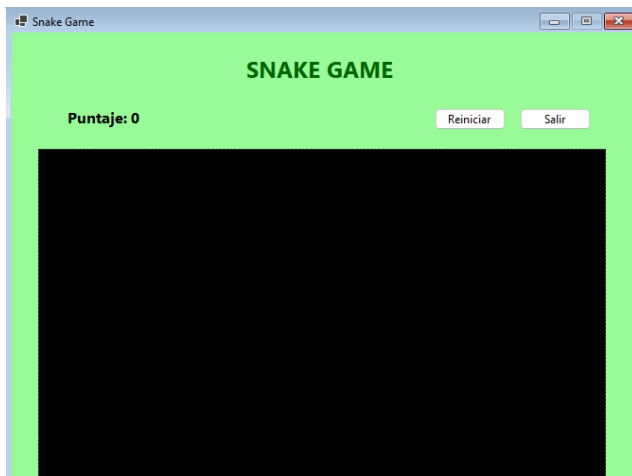
Desarrollar un videojuego tipo Snake en lenguaje C# utilizando programación orientada a objetos e interfaz gráfica, aplicando estructuras de control, colecciones y manejo de eventos para lograr un funcionamiento interactivo y dinámico.

OBJETIVOS ESPECÍFICOS

- Diseñar una interfaz gráfica funcional para el videojuego utilizando Windows Forms.
- Implementar el movimiento de la serpiente mediante el uso del teclado.
- Generar alimentos en posiciones aleatorias dentro del tablero de juego.
- Incrementar el tamaño de la serpiente al consumir alimentos.
- Implementar un sistema de puntaje durante la partida.
- Detectar colisiones contra los bordes y contra el propio cuerpo de la serpiente.
- Aplicar principios de Programación Orientada a Objetos mediante clases, métodos, encapsulamiento e interfaces.
- Utilizar colecciones para almacenar y gestionar el cuerpo de la serpiente.

- Implementar mensajes de “Game Over” y condición de victoria dentro del juego.

INTERFAZ



CÓDIGO

- Clase Snake: Representa la serpiente.

```

1  using System;
2  using System.Collections.Generic;
3  using System.Drawing;
4
5  namespace Snake_YordanCohaila
6  {
7      3 referencias
8      public class Snake : IDibujable
9      {
10         14 referencias
11         public List<Point> Cuerpo { get; set; }
12
13         9 referencias
14         public int DX { get; set; }
15         9 referencias
16         public int DY { get; set; }
17
18         9 referencias
19         public int TamañoCelda { get; set; }
20
21         1 referencia
22         public Snake()
23         {
24             TamañoCelda = 20;
25
26             Cuerpo = new List<Point>();
27
28             Cuerpo.Add(new Point(5, 5));
29             Cuerpo.Add(new Point(4, 5));
30             Cuerpo.Add(new Point(3, 5));
31
32             DX = 1;
33             DY = 0;
34         }
35
36         1 referencia
37         public void Mover(bool crecer)
38         {
39             Point cabeza = Cuerpo[0];
40
41             Point nuevaCabeza = new Point(
42                 cabeza.X + DX,
43                 cabeza.Y + DY
44             );
45
46             Cuerpo.Insert(0, nuevaCabeza);
47
48             if (!crecer)
49             {
50                 Cuerpo.RemoveAt(Cuerpo.Count - 1);
51             }
52         }
53
54         2 referencias
55         public void Dibujar(Graphics g)
56         {

```

```

49         foreach (Point parte in Cuerpo)
50         {
51             g.FillRectangle(
52                 Brushes.Lime,
53                 parte.X * TamañoCelda,
54                 parte.Y * TamañoCelda,
55                 TamañoCelda,
56                 TamañoCelda
57             );
58         }
59     }
60 }

```

- Clase Food: Representa la comida

```

1  using System;
2  using System.Drawing;
3
4  namespace Snake_YordanCohaila
5  {
6      3 referencias
7      public class Food : IDibujable
8      {
9          4 referencias
10         public Point Posicion { get; set; }
11
12         7 referencias
13         public int TamañoCelda { get; set; }
14
15         Random rnd = new Random();
16
17         1 referencia
18         public Food()
19         {
20             TamañoCelda = 20;
21         }
22
23         2 referencias
24         public void Generar(
25             int ancho,
26             int alto,
27             List<Point> cuerpoSnake
28         )
29         {
30             List<Point> espaciosLibres =
31                 new List<Point>();
32
33             int columnas = ancho / TamañoCelda;
34             int filas = alto / TamañoCelda;
35
36             // Buscar todos los espacios libres
37             for (int x = 0; x < columnas; x++)
38             {
39                 for (int y = 0; y < filas; y++)
40                 {
41                     Point p = new Point(x, y);
42
43                     bool ocupado = false;
44
45                     foreach (Point parte in cuerpoSnake)
46                     {
47                         if (parte == p)
48                         {
49                             ocupado = true;
50                             break;
51                         }
52                     }
53
54                     if (!ocupado)
55                     {
56                         espaciosLibres.Add(p);
57                     }
58                 }
59             }
60         }
61     }
62 }

```

```

53     }
54 }
55
56 // Si no hay espacios libres
57 if (espaciosLibres.Count == 0)
58     return;
59
60 // Elegir posición aleatoria válida
61 int indice = rnd.Next(espaciosLibres.Count);
62
63 Posicion = espaciosLibres[indice];
64 }
65
66 2 referencias
67 public void Dibujar(Graphics g)
68 {
69     g.FillEllipse(
70         Brushes.Red,
71         Posicion.X * TamañoCelda,
72         Posicion.Y * TamañoCelda,
73         TamañoCelda,
74         TamañoCelda
75     );
76 }
77 }

```

- Clase GameManager: Controla lógica, puntaje, colisiones, etc.

```

1  using System.Drawing;
2
3  namespace Snake_YordanCohaila
4  {
5      4 referencias
6      public class GameManager
7      {
8          public Snake snake;
9          public Food food;
10
11          4 referencias
12          public int Puntaje { get; set; }
13
14          5 referencias
15          public bool GameOver { get; set; }
16
17          3 referencias
18          public bool Victoria { get; set; }
19
20          int ancho;
21          int alto;
22
23          2 referencias
24          public GameManager(int anchoPanel, int altoPanel)
25          {
26              bool puedeCambiarDireccion = true;
27
28              ancho = anchoPanel;
29              alto = altoPanel;
30
31              snake = new Snake();
32
33              food = new Food();
34
35              food.Generar(
36                  ancho,
37                  alto,
38                  snake.Cuerpo
39              );
40
41              Puntaje = 0;
42
43              GameOver = false;
44
45              Victoria = false;
46          }
47
48          1 referencia
49          public void Actualizar()
50          {
51              if (GameOver)
52                  return;
53
54              Point cabeza = snake.Cuerpo[0];
55
56              Point siguiente = new Point(
57                  cabeza.X + snake.DX,
58

```

```

52         cabeza.Y + snake.OY
53     );
54
55     // COLISIÓN CON PAREDES
56
57     int maxX = ancho / snake.TamañoCelda;
58     int maxY = alto / snake.TamañoCelda;
59
60
61     if (siguiente.X < 0 ||
62         siguiente.Y < 0 ||
63         siguiente.X >= maxX ||
64         siguiente.Y >= maxY)
65     {
66         GameOver = true;
67         return;
68     }
69
70     // COLISIÓN CONSIGO MISMA
71
72     foreach (Point parte in snake.Cuerpo)
73     {
74         if (siguiente == parte)
75         {
76             GameOver = true;
77             return;
78         }
79     }
80
81     // COMER
82
83     bool comier =
84         siguiente == food.Posicion;
85
86     snake.Mover(comier);
87
88     if (comier)
89     {
90         Puntaje++;
91
92         int totalCeldas =
93             (ancho / snake.TamañoCelda) *
94             (alto / snake.TamañoCelda);
95
96         if (snake.Cuerpo.Count >= totalCeldas)
97         {
98             Victoria = true;
99             return;
100         }
101
102         food.Generar(
103             ancho,
104             alto,
105             snake.Cuerpo

```

```

106         );
107     }
108 }
109 }
110 }

```

- Interfaz IDibujable

```

1 using System.Drawing;
2
3 2 referencias
4 interface IDibujable
5 {
6     4 referencias
7     void Dibujar(Graphics g);
8 }

```

FORM PRINCIPAL

```
1  using Snake_YordanCohaila;
2  using System.Collections.Generic;
3  using System.Drawing;
4
5  namespace Snake_YordanCohaila
6  {
7      3 referencias
8      public partial class Form1 : Form
9      {
10         GameManager game;
11
12         bool puedeCambiarDireccion = true;
13
14         // Tamaño de cada bloque
15         int tamañoCelda = 20;
16
17         // Dirección inicial
18         int dx = 1;
19         int dy = 0;
20
21         // Serpiente
22         List<Point> snake = new List<Point>();
23
24         // Comida
25         Point comida;
26
27         // Puntaje
28         int puntaje = 0;
29
30         // Random
31         Random rnd = new Random();
32         1 referencia
33         public Form1()
34         {
35             InitializeComponent();
36
37             game = new GameManager(
38                 pnlJuego.Width,
39                 pnlJuego.Height
40             );
41
42             this.KeyPreview = true;
43
44             this.ActiveControl = null;
45
46             timerJuego.Start();
47         }
48         0 referencias
49         private void IniciarJuego()
50         {
51             snake.Clear();
```

```

49
50 // Posición inicial serpiente
51 snake.Add(new Point(5, 5));
52 snake.Add(new Point(4, 5));
53 snake.Add(new Point(3, 5));
54
55 // Dirección inicial
56 dx = 1;
57 dy = 0;
58
59 // Puntaje
60 puntaje = 0;
61 lblPuntaje.Text = "Puntaje: 0";
62
63 // Generar comida
64 GenerarComida();
65
66 // Iniciar timer
67 timerJuego.Start();
68
69 // Redibujar
70 pnlJuego.Invalidate();
71 }
72
73 2 referencias
74 private void GenerarComida()
75 {
76     int x = rnd.Next(0, pnlJuego.Width / tamañoCelda);
77     int y = rnd.Next(0, pnlJuego.Height / tamañoCelda);
78
79     comida = new Point(x, y);
80 }
81
82 1 referencia
83 private void pnlJuego_Paint(object sender, PaintEventArgs e)
84 {
85     game.snake.Dibujar(e.Graphics);
86
87     game.food.Dibujar(e.Graphics);
88 }
89
90 1 referencia
91 private void timerJuego_Tick(object sender, EventArgs e)
92 {
93     game.Actualizar();
94
95     puedeCambiarDireccion = true;
96
97     lblPuntaje.Text =
98         "Puntaje: " + game.Puntaje;

```



```

95
96     pnlJuego.Invalidate();
97
98     if (game.Victoria)
99     {
100         timerJuego.Stop();
101
102         MessageBox.Show(
103             ";GANASTE!\nLa serpiente llenó todo el tablero"
104         );
105
106         return;
107     }
108
109     if (game.GameOver)
110     {
111         timerJuego.Stop();
112
113         MessageBox.Show(
114             "Game Over\nPuntaje: " + game.Puntaje
115         );
116     }
117 }
118 0 referencias
119 private void MoverSnake()
120 {
121     Point cabeza = snake[0];
122
123     Point nuevaCabeza = new Point(
124         cabeza.X + dx,
125         cabeza.Y + dy
126     );
127
128     // Insertar nueva cabeza
129     snake.Insert(0, nuevaCabeza);
130
131     // Comer comida
132     if (nuevaCabeza == comida)
133     {
134         puntaje++;
135         lblPuntaje.Text = "Puntaje: " + puntaje;
136
137         GenerarComida();
138     }
139     else
140     {
141         // Eliminar cola
142         snake.RemoveAt(snake.Count - 1);

```

```

142     }
143 }
144
145 1 referencia
146 private void Form1_KeyDown(object sender, KeyEventArgs e)
147 {
148     if (!puedeCambiarDireccion)
149         return;
150
151     if (e.KeyCode == Keys.W && game.snake.DY != 1)
152     {
153         game.snake.DX = 0;
154         game.snake.DY = -1;
155
156         puedeCambiarDireccion = false;
157     }
158     else if (e.KeyCode == Keys.S && game.snake.DY != -1)
159     {
160         game.snake.DX = 0;
161         game.snake.DY = 1;
162
163         puedeCambiarDireccion = false;
164     }
165     else if (e.KeyCode == Keys.A && game.snake.DX != 1)
166     {
167         game.snake.DX = -1;
168         game.snake.DY = 0;
169
170         puedeCambiarDireccion = false;
171     }
172     else if (e.KeyCode == Keys.D && game.snake.DX != -1)
173     {
174         game.snake.DX = 1;
175         game.snake.DY = 0;
176
177         puedeCambiarDireccion = false;
178     }
179 }
180 1 referencia
181 private void Form1_Load(object sender, EventArgs e)
182 {
183 }
184
185 1 referencia
186 private void btnReiniciar_Click(object sender, EventArgs e)
187 {
188     game = new GameManager(
189         pnlJuego.Width,

```

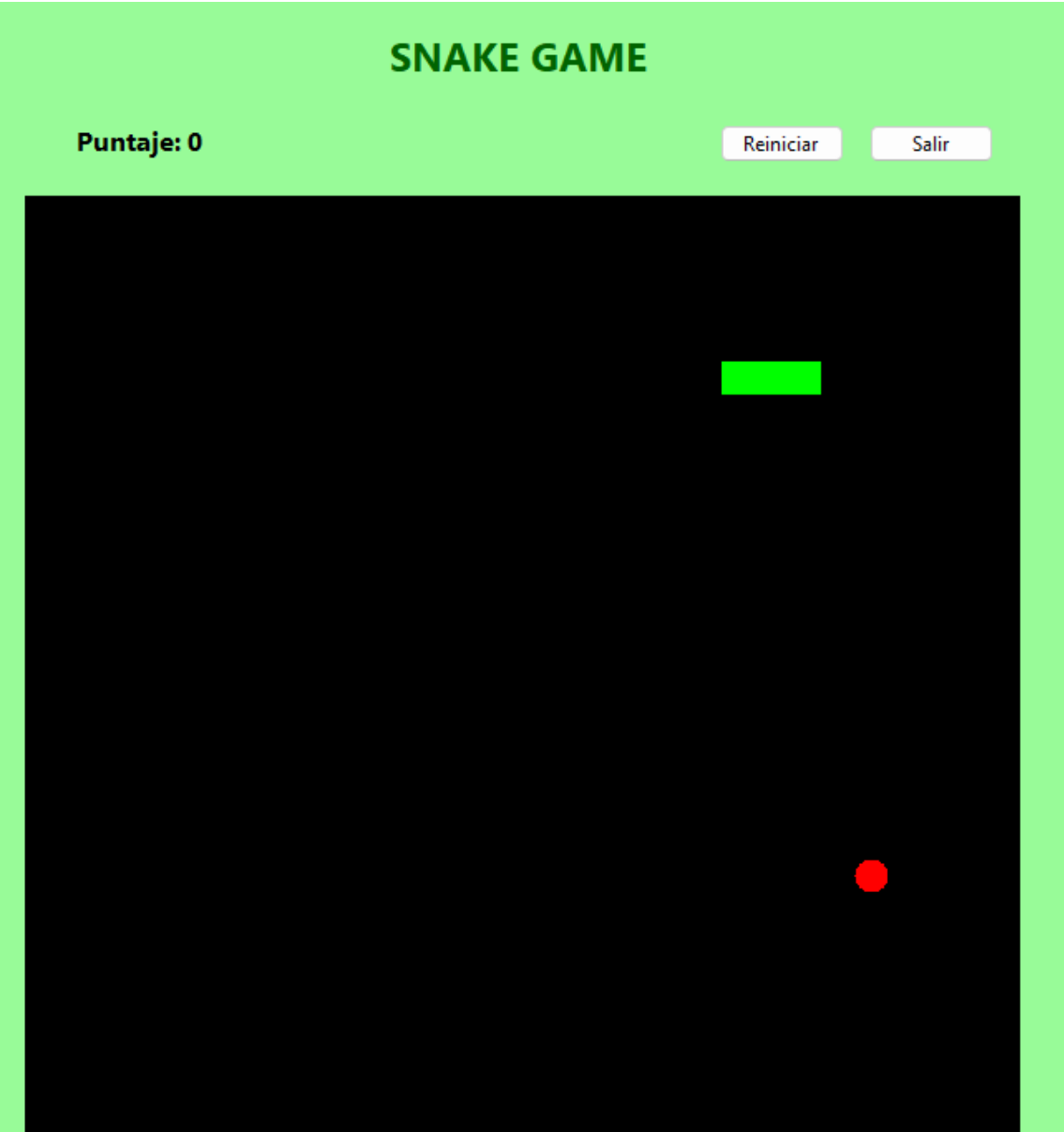
```

188         pnlJuego.Height
189     );
190
191     lblPuntaje.Text = "Puntaje: 0";
192
193     timerJuego.Start();
194
195     pnlJuego.Invalidate();
196 }
197
198 1 referencia
199 private void btnSalir_Click(object sender, EventArgs e)
200 {
201     Application.Exit();
202 }
203
204 private void btnSalir_KeyDown(object sender, KeyEventArgs e)
205 {
206 }
207 }
208 }

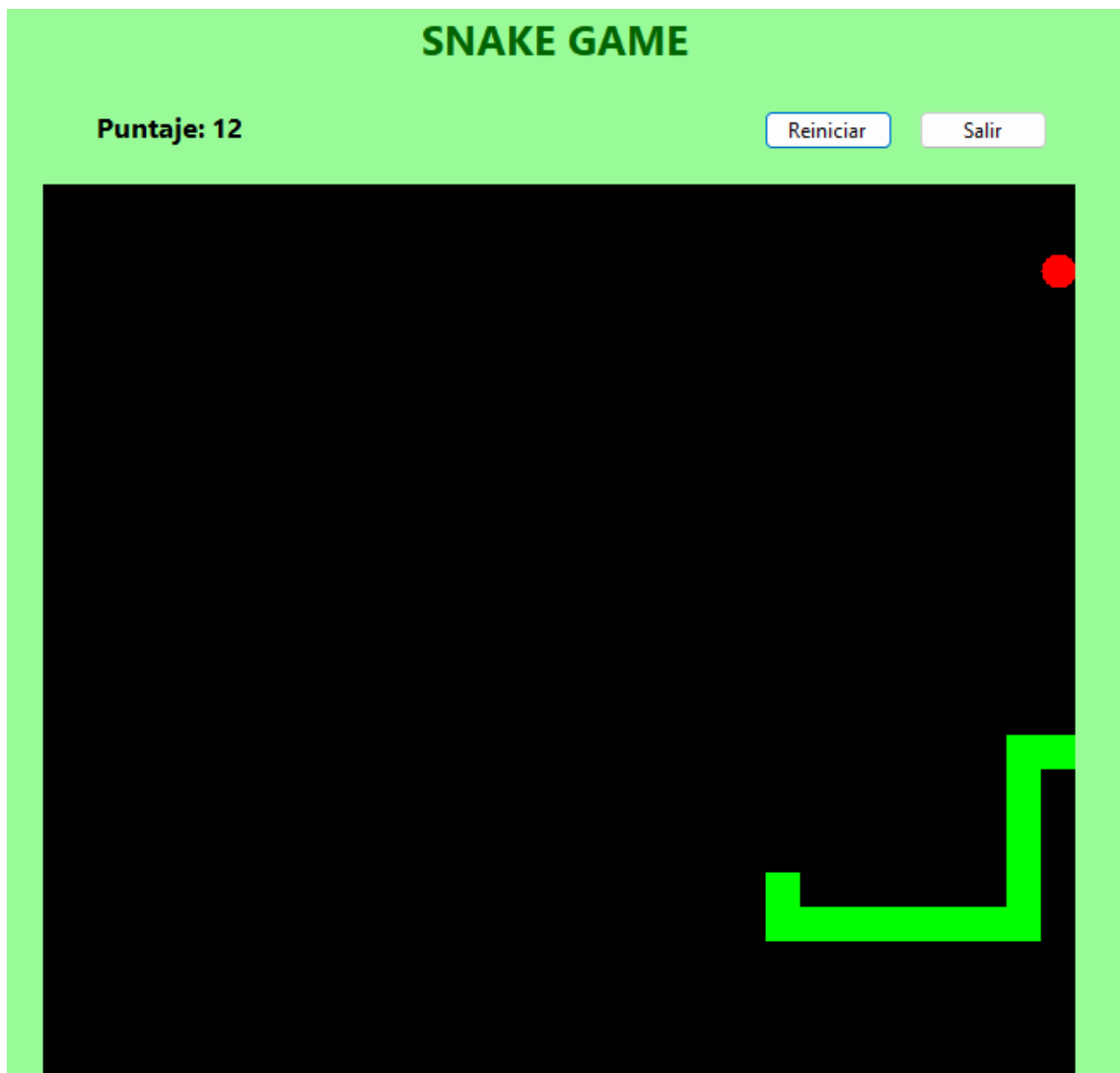
```

EJECUCIÓN

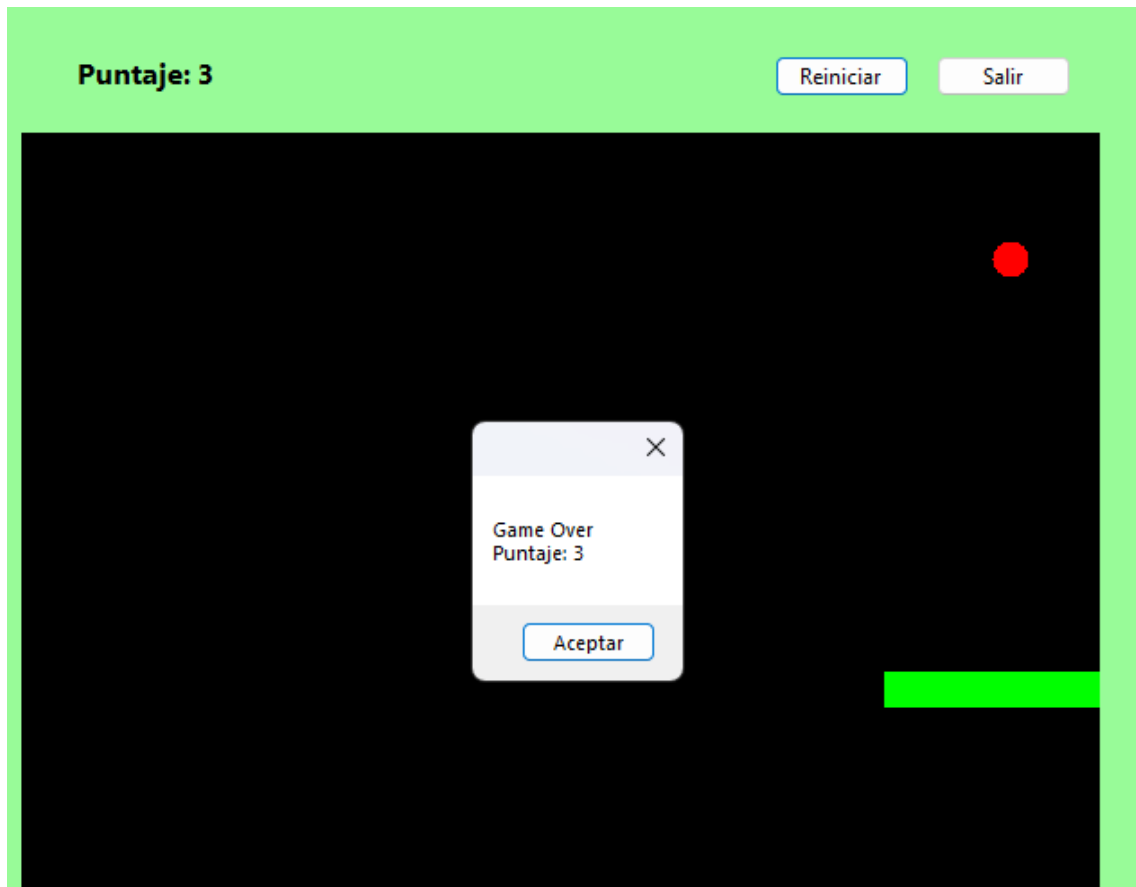
- Posición inicial



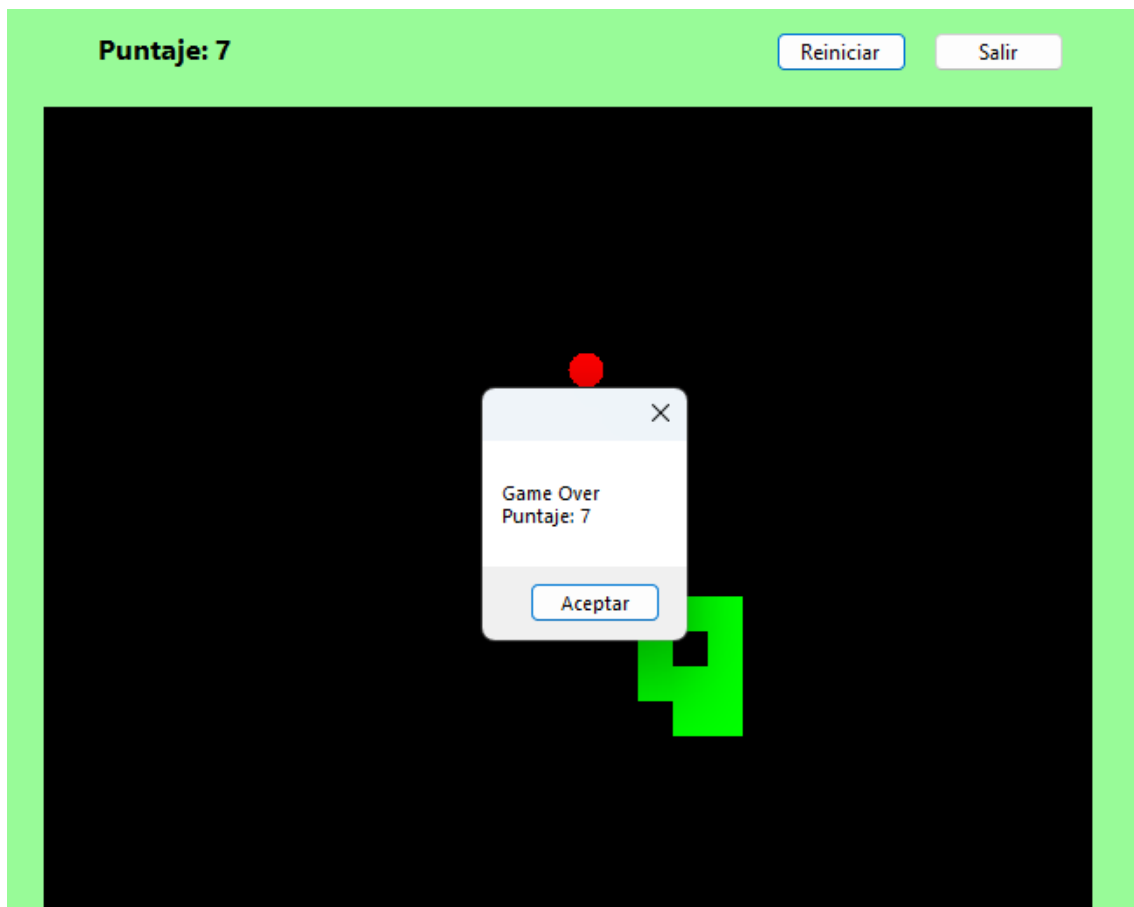
- Acumulando puntaje



- Game Over (al chocar contra una pared)



- Game Over (al chocar con su propio cuerpo)



CONCLUSIONES

- Se logró desarrollar correctamente un videojuego tipo Snake funcional utilizando el lenguaje C# y Windows Forms.
- El proyecto permitió aplicar de manera práctica los principios fundamentales de la Programación Orientada a Objetos, especialmente el uso de clases, objetos, interfaces y encapsulamiento.
- El uso de colecciones facilitó el manejo dinámico del cuerpo de la serpiente durante el desarrollo del juego.
- La implementación de eventos y temporizadores permitió actualizar continuamente el estado del juego y controlar el movimiento en tiempo real.
- Se consiguió implementar adecuadamente las principales funcionalidades del videojuego, como movimiento, generación de comida, puntaje, colisiones y reinicio de partida.
- El desarrollo de este proyecto contribuyó al fortalecimiento de habilidades en lógica de programación, diseño de interfaces gráficas y organización estructurada del código fuente.